Runlevels vs targets

Systemd tiene el concepto de target para reemplazar el concepto clásico de runlevel de los sistemas sysvinit. Un target es similar a un runlevel, con algunas diferencias. Algunos targets incluso se crean y heredan los servicios de otro target, y a su vez, puede agregar servicios nuevos… esto nos recordará al concepto de generalización y herencia de la POO.

Un target tiene un nombre, no un número como los runlevels, y está orientado a un uso específico.

Al ser un concepto equivalente al de runlevel, se puede utilizar init o telinit para intercambiar los targets en el sistema, tal cual se hacía en sysvinit con los números de runlevel.

Los targets son directorios que se encuentran generalmente en **/etc/systemd/system/** y tienen nombres similares a "mitarget.target.wants". Dentro del directorio de un target se linkean como symlink (ln -s) los archivos de servicio que se encuentran por lo general en **/lib/systemd/system/** y son archivos de servicios con "extensión" **.service.**

A estos archivos se los denomina "archivos de unidad de servicio".

Tabla de equivalencias SysVinit y SystemD

En la siguiente tabla presento un resumen breve de equivalencias de comandos entre sysvinit y systemd:

Comando SysVinit	Comando Systemd	Descripción
service miservicio start	systemctl start miservicio	Reiniciar un servicio (no es persistente al reinicio)
service miservicio stop	systemctl stop miservicio	Detiene un servicio (no es persistente al reinicio)
service miservicio restart	systemctl restart miservicio	Detiene un servicio y lo inicia nuevamente
service miservicio reload	systemctl reload miservicio	Recarga la configuración de un servicio sin detenerlo. Depende del servicio si soporta esta opción o no.
service miservicio condrestart	systemctl condrestart miservicio	Reinicia el servicio si ya se encuentra corriendo.

Comando SysVinit	Comando Systemd	Descripción
service miservicio status	systemctl status miservicio	Muestra el estado de ejecución de un servicio.
ls /etc/rc.d/init.d/	systemctl (ó) systemctl list-unit-files --type=service (ó) ls /lib/systemd/system/*.service /etc/systemd/system/*.service	Lista los servicios disponibles en el sistema.
chkconfig miservicio on	systemctl enable miservicio	Activa el servicio para iniciarse al próximo inicio del sistema.
chkconfig miservicio off	systemctl disable miservicio	Desactiva el servicio para iniciarse al próximo inicio del sistema.
chkconfig miservicio	systemctl is-enabled miservicio	Verifica si el servicio está activo o no al inicio del sistema operativo.
chkconfig --list	systemctl list-unit-files --type=service (ó) ls /etc/systemd/system/*.wants/	Imprime una lista de los servicios del sistema, indicando su estado de activación y runlevels para los que corre.
chkconfig miservicio --list	ls /etc/systemd/system/*.wants/miservicio.service	Lista para qué runlevel está activo o no un servicio.
chkconfig miservicio --add	systemctl daemon-reload	Crea un nuevo archivo de servicio, o modifica uno existente.

En la siguiente imagen se ve un resumen interesante de estas equivalencias y comandos, y varias más de las que agregué en la tabla. (Click en la imagen para ver en tamaño completo)

Systemd vs SysVinit

Systemd Commands: http://linoxide.com/linux-command/linux-systemd-commands/

Service Related Commands

Comments	SysVinit	Systemd
Start a service	service dummy start	systemctl start dummy.service
Stop a service	service dummy stop	systemctl stop dummy.service
Restart a service	service dummy restart	systemctl restart dummy.service
Reload a service	service dummy reload	systemctl reload dummy.service
Service status	service dummy status	systemctl status dummy.service
Restart a service if already running	service dummy condrestart	systemctl condrestart dummy.service
Enable service at startup	chkconfig dummy on	systemctl enable dummy.service
Disable service at startup	chkconfig dummy off	systemctl disable dummy.service
Check if a service is enabled at startup	chkconfig dummy	systemctl is-enabled dummy.service
Create a new service file or modify configuration	chkconfig dummy --add	systemctl daemon-reload

Note : New version of systemd support "systemctl start dummy" format.

Runlevels

Comments	SysVinit	Systemd
System halt	0	runlevel0.target, poweroff.target
Single user mode	1, s, single	runlevel1.target, rescue.target
Multi user	2	runlevel2.target, multi-user.target
Multi user with Network	3	runlevel3.target, multi-user.target
Experimental	4	runlevel4.target, multi-user.target
Multi user, with network, graphical mode	5	runlevel5.target, graphical.target
Reboot	6	runlevel6.target, reboot.target
Emergency Shell	emergency	emergency.target
Change to multi user runlevel/target	telinit 3	systemctl isolate multi-user.target (OR systemctl isolate runlevel3. target)
Set multi-user target on next boot	sed s/^id:.*:initdefault:/ id:3:initdefault:/	ln -sf /lib/systemd/system/multi-user.target /etc/systemd/system/ default.target
Check current runlevel	runlevel	systemctl get-default
Change default runlevel	sed s/^id:.*:initdefault:/ id:3:initdefault:/	systemctl set-default multi-user.target

Miscellaneous Commands

Comments	SysVinit	Systemd
System halt	halt	systemctl halt
Power off the system	poweroff	systemctl poweroff
Restart the system	reboot	systemctl reboot
Suspend the system	pm-suspend	systemctl suspend
Hibernate	pm-hibernate	systemctl hibernate
Follow the system log file	tail -f /var/log/messages or tail -f /var/log/syslog	journalctl -f

Systemd New Commands

Comments	Systemd
Execute a systemd command on remote host	systemctl dummy.service start -H user@host
Check boot time	systemd-analyze or systemd-analyze time
Kill all processes related to a service	systemctl kill dummy
Get logs for events for today	journalctl --since=today
Hostname and other host related information	hostnamectl
Date and time of system with timezone and other information	timedatectl

Brought to you by LinOxide Team

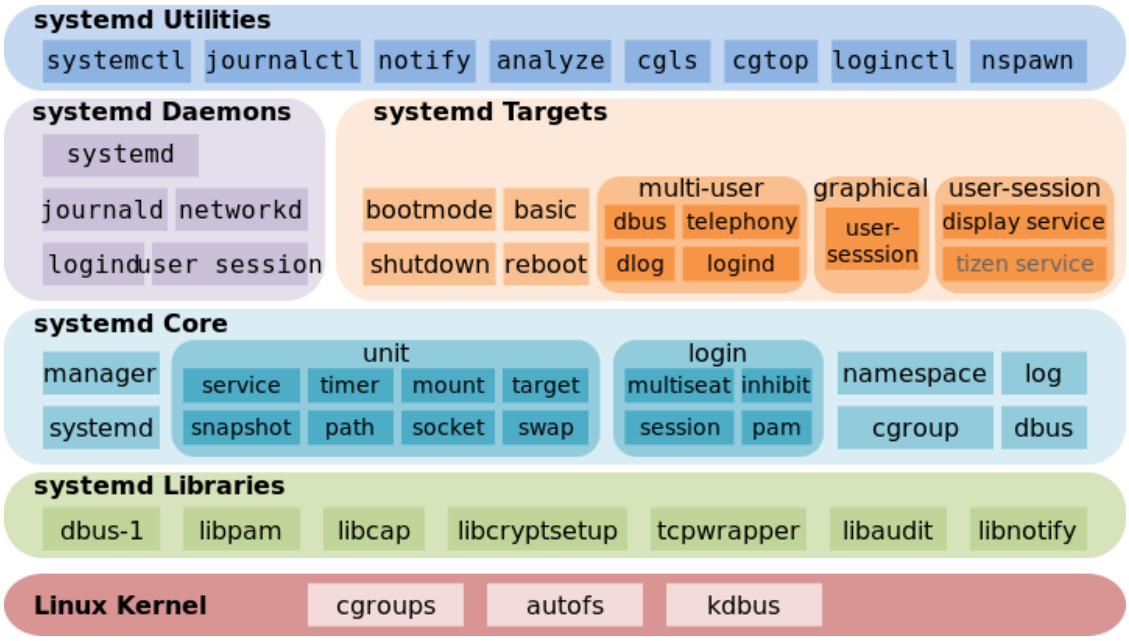
LinOxide Linux Freedom Thoughts

systemd: ejecutando un script al inicio de GNU/Linux

Cuántas veces no hemos necesitado ejecutar alguna aplicación o servicio al iniciar nuestra distro GNU/Linux?

Para los que venimos del viejo y querido SysVInit, todo se reducía a crear un archivo en **/etc/init.d/** con una determinada estructura, y activarlo.

Ahora, con systemd la cosa parece que se complica un poco, pero en realidad no tanto, y hasta resulta un poco más ordenado que el viejo sistema.



El script / binario / ejecutable

En este ejemplo vamos a suponer que necesitamos ejecutar un script que hemos colocado en **/usr/local/bin/miscript.sh** (nombre genérico)… por supuesto cada uno podrá adaptarlo a sus necesidades.

Cualquier archivo ejecutable, ya sea **script** o **binario,** vamos a poder cargarlo en nuestro sistema al momento de iniciar el sistema operativo.

Por supuesto, el script o binario deberá tener permisos de ejecución, y para ellos podemos correr un comando similar a este:

chmod +x /usr/local/bin/miscript.sh

Unidad de servicio systemd

Luego tenemos que crear una **unidad de servicio systemd** (*systemd serivce unit*).

Vamos a verlo funcionando con un ejemplo súper sencillo:

```
[Unit]
Description=<em class="replaceable">Mi script automatico</em>
After=network.target

[Service]
ExecStart=<em>/usr/local/bin/miscript.sh</em>
Type=forking
PIDFile=<em class="replaceable">path_to_pidfile</em>

[Install]
WantedBy=default.target
```

Donde:

- **Description**: información descriptiva que es mostrada en el log de journal de systemd cuando ejecutamos "**systemctl status servicio.service**"
- **After:** indica el servicio luego del cual correrá nuestro script. Supongamos que el script debe realizar algunas operaciones de red… sería lógico ejecutarlo luego de que se haya corrido **network.service**, cierto?
  Se pueden agregar otros servicios separándolos por un espacio en blanco.
- **ExecStart:** permite especificar la ruta completa al script que vamos a ejecutar, el nuestro.
- **Type**: forking en este caso es utilizado por daemons o residentes para que realicen una clonación de la llamada al sistema. El proceso principal del servicio es creado con un **pid** especificado en el archivo de pid escrito a continuación.
- **WantedBy:** especifica dentro de qué "target" de systemd debe ser instalado. Dejémoslo en default.target.

Sacando las opciones de *Type*, *PIDFile* y *Description*, esto es lo mínimo indispensable para poder ejecutar un script en systemd, las líneas mínimas para un *service unit*.

¿Que queremos más configuraciones y adaptaciones para nuestro entorno? Nada mejor que acercarse por la *mesa de ayuda*:

man systemd.service

Luego debemos darle permisos al nuevo archivo .service:

chmod 0644 /etc/systemd/system/miscript.service

Luego de crear nuestra unidad de servicio, o de modificarla en el futuro, siempre debemos ejecutar el comando:

systemctl daemon-reload

Habilitando y ejecutando

Con estos pasos listos, ahora podremos habilitar el servicio con:

systemctl enable miscript.service

Y ejecutarlo con:

systemctl start miscript.service

Al estar habilitado / enabled, y al haberlo “instalado” en el target por defecto (default.target en la última línea del ejemplo), podremos ver su enlace simbólico en /etc/systemd/system/default.target.wants/

$ ls -l /etc/systemd/system/default.target.wants/
total 0
lrwxrwxrwx 1 root root 36 Mar 18 21:12 miscript.service ->
/etc/systemd/system/miscript.service

Y por supuesto, si ejecutamos:

systemctl disable miscript.service

Estaremos eliminando dicho enlace simbólico.